

Perlmutter storage benchmark

Pilot analysis

Manoj Khatri / HPCAI-lab | 2 October 2026

Storage placement and concurrency interact. Native SQLite metadata writes gain 1.85x from tmpfs at one client, but the observed gain is 0.97x at 64 clients. PostgreSQL and InfluxDB follow different scaling patterns. These are pilot observations from one allocation per deployment and workload.

Verified analysis coverage

Deployments	Workloads and conditions	Observations
SQLite native, SQLite Shifter, PostgreSQL, InfluxDB	Metadata and telemetry; Lustre and tmpfs; 1, 16, 64 clients	240 measured + 48 warmups; 48 conditions; 5 measured repetitions each

Each trial wrote 1,000,000 records in batches of 1,000. SQL pilots issued 64,000 range queries; InfluxDB issued 1,024. Each query returned 100 rows. PostgreSQL and InfluxDB used native Python clients with a Shifter server.

Write throughput: tmpfs / Lustre

Deployment	Metadata: 1 client	Metadata: 64	Telemetry: 1	Telemetry: 64
SQLite native	1.85x	0.97x	1.52x	1.20x
SQLite Shifter	1.76x	1.05x	1.48x	1.14x
PostgreSQL	1.42x	1.36x	1.37x	1.26x
InfluxDB	1.13x	1.11x	1.10x	1.03x

Ratios above use arithmetic mean throughput. SQL write completion includes the final checkpoint. InfluxDB write completion ends at HTTP acknowledgement, so these ratios characterize each deployment; they do not establish a durability-matched engine ranking.

Three findings worth carrying forward

SQLite metadata: native tmpfs write throughput falls from 442.47 to 192.20 krecords/s as clients increase from 1 to 64. Its 64-client storage-gain interval spans parity; the mean alone does not establish that Lustre is faster.

Concurrency: PostgreSQL metadata writes on Lustre reach 5.39x the one-client throughput at 64 clients. InfluxDB reaches 9.60x for metadata and 12.57x for telemetry under its HTTP-acknowledgement boundary.

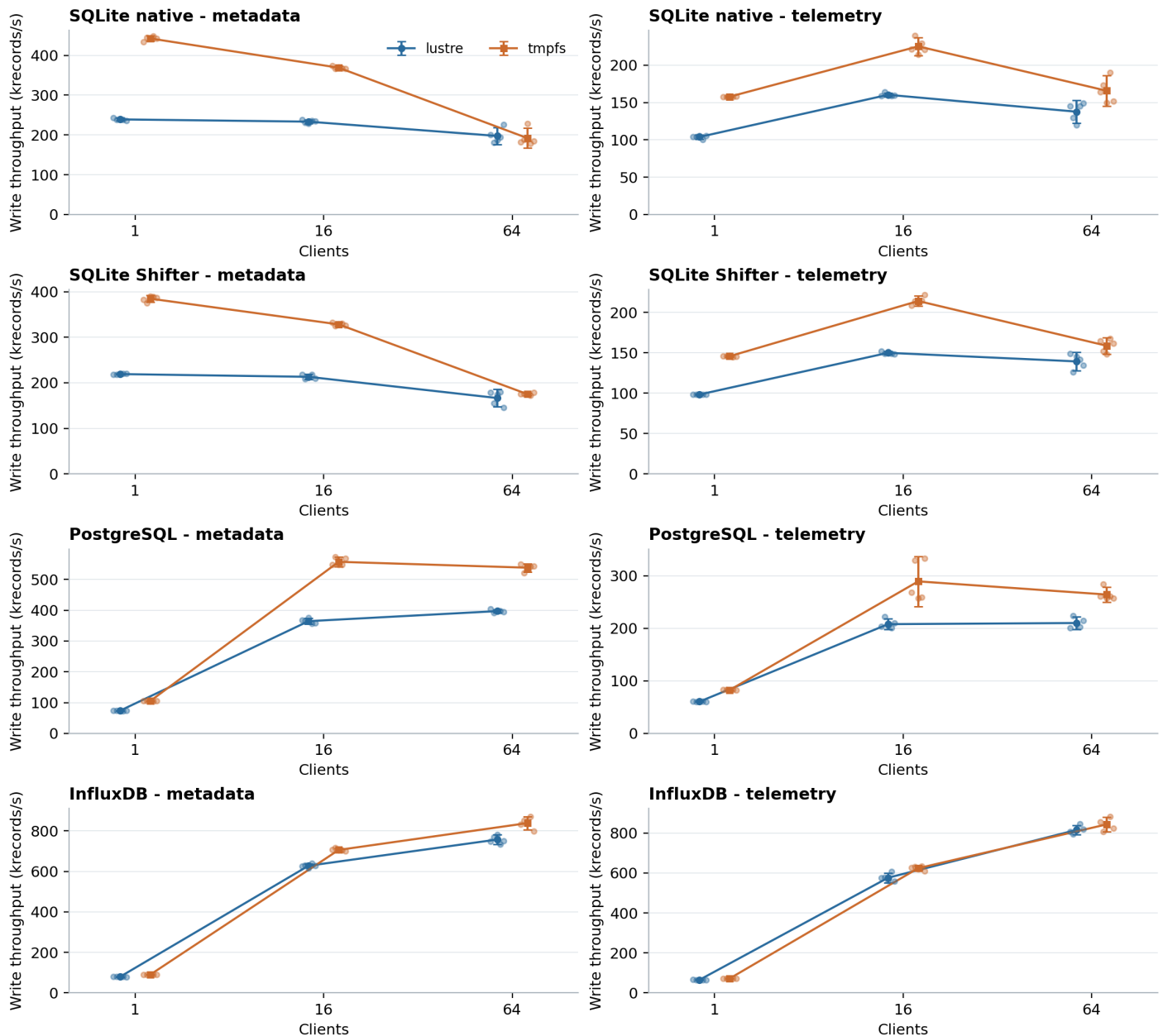
Read variability: InfluxDB telemetry at 16 clients on Lustre has 39.2% sample CV. Two repetitions deliver about 192-196 queries/s, and three deliver 432-446. All five observations are retained.

Status: reproducible pilot analysis complete. Independent allocation repeats, matched comparison protocols, mixed workloads and placement-model evaluation remain.

Write throughput and concurrency

Points are the five measured trials. Lines are arithmetic means; whiskers are pointwise 95% t intervals using within-allocation repetitions. Panel scales differ.

Write completion throughput



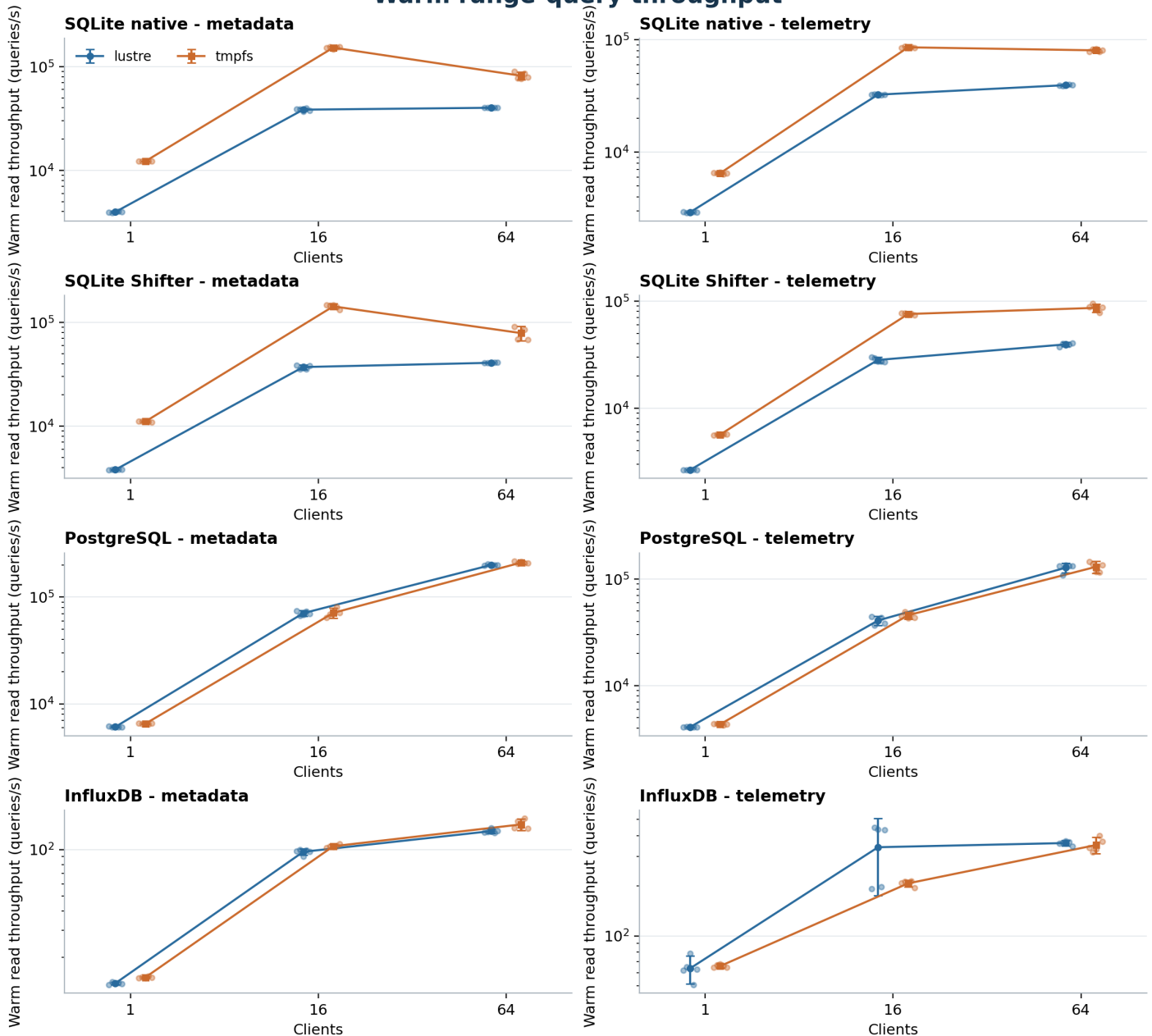
Measurement boundary: SQLite and PostgreSQL include final checkpoint completion. InfluxDB includes HTTP acknowledgement and excludes later server shutdown. Both client-side preparation and execution behavior can influence the measured phase.

Increasing clients reduces metadata write throughput for SQLite. PostgreSQL shows its largest gains by 16 clients, with workload/storage-dependent behavior at 64. InfluxDB mean write throughput rises through 64 clients in both workloads.

Warm read throughput

Logarithmic y axes show queries/s. Points are individual trials; whiskers are pointwise 95% t intervals around arithmetic means. These are warm reads after writes and validation, not cold storage reads.

Warm range-query throughput



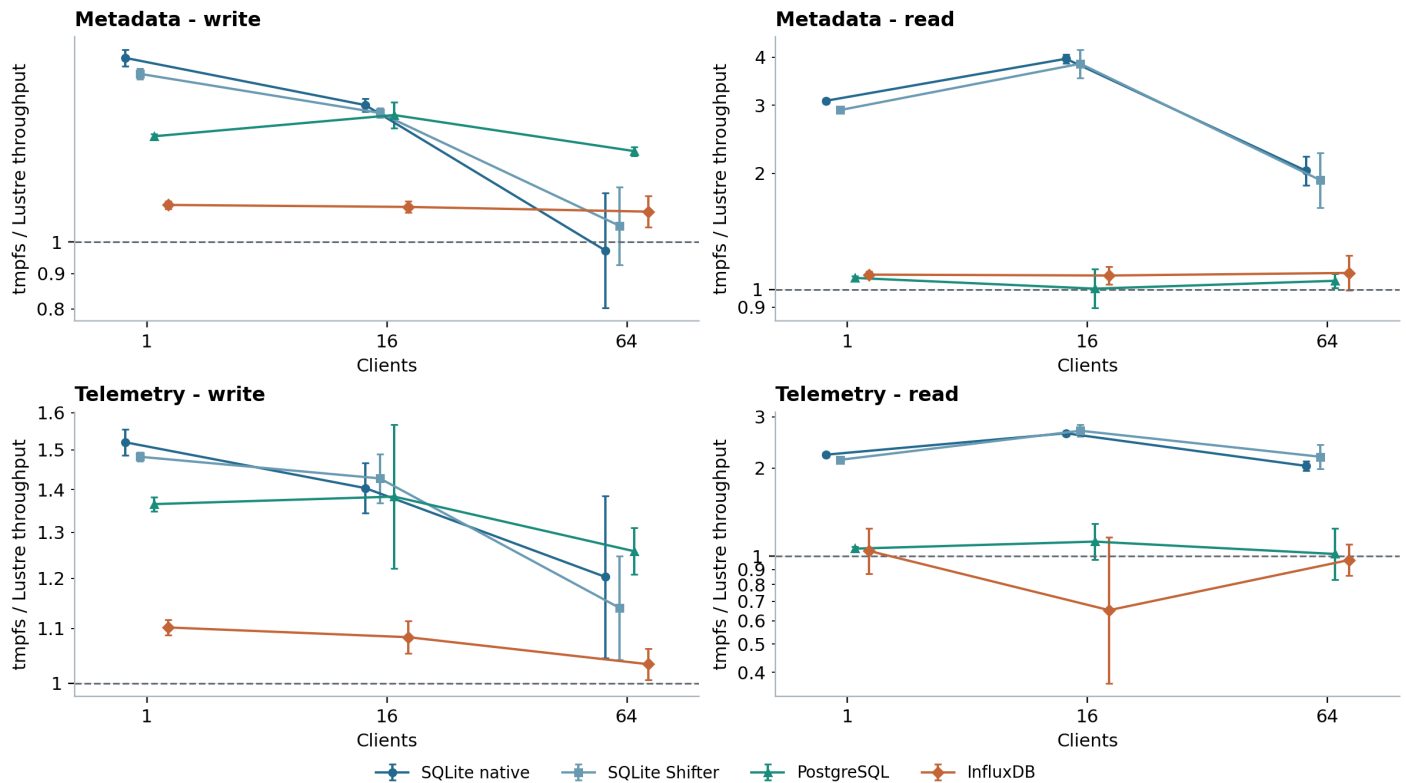
SQL pilots ran 64,000 queries per trial; InfluxDB ran 1,024. Query translation, client protocol, engine versions and physical schemas differ. Per-second normalization alone does not remove these protocol and duration differences.

Native SQLite metadata reads on tmpfs peak at 16 among the tested client counts: 152.22 kqueries/s versus 81.51 at 64. PostgreSQL metadata reads on Lustre rise from 6.08 to 200.12 kqueries/s between 1 and 64 clients.

Storage gains and interaction contrasts

Plot points are geometric means of repetition-matched throughput ratios. Whiskers are pointwise 95% t intervals on log ratios. These estimates differ slightly from ratios of arithmetic means on page 1.

Storage gains within each deployment



How much does the storage gain change from 1 to 64 clients?

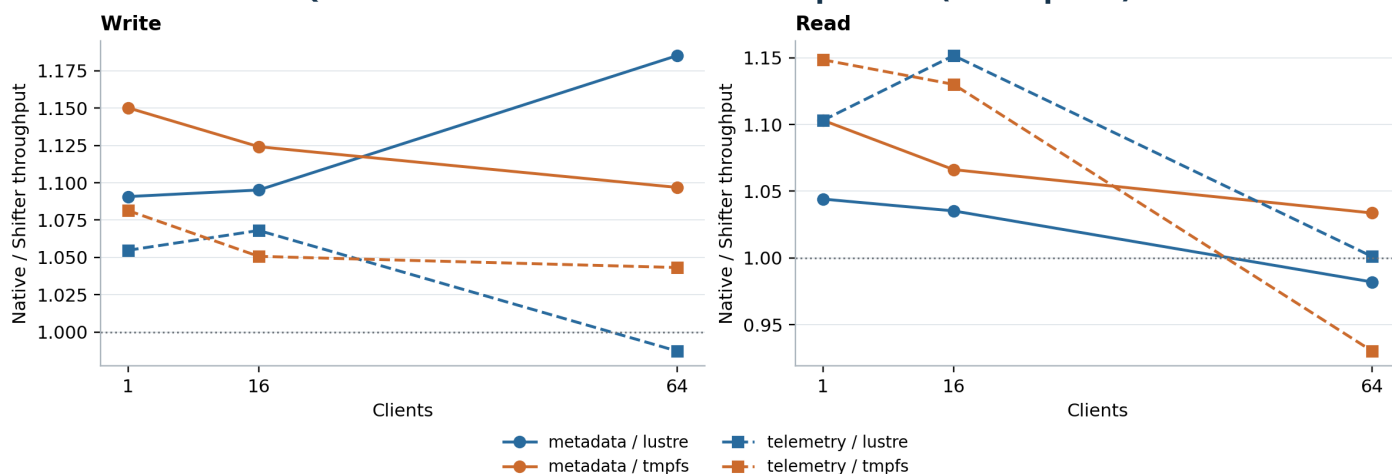
Interaction contrast $I = (\text{tmpfs/Lustre at 64 clients}) / (\text{tmpfs/Lustre at 1 client})$. A value below 1 means the relative tmpfs advantage shrinks. The table uses geometric means of five block contrasts.

Deployment	Metadata write I [95% CI]	Telemetry write I [95% CI]
SQLite native	0.525 [0.432, 0.639]	0.792 [0.695, 0.902]
SQLite Shifter	0.601 [0.522, 0.692]	0.769 [0.700, 0.845]
PostgreSQL	0.952 [0.939, 0.964]	0.921 [0.887, 0.957]
InfluxDB	0.978 [0.929, 1.029]	0.938 [0.908, 0.969]

Native SQLite metadata has the strongest reduction: $I = 0.525$ [0.432, 0.639]. InfluxDB metadata is closer to unchanged: 0.978 [0.929, 1.029]. These are exploratory contrasts, not an omnibus interaction test or evidence across independent allocations.

Runtime comparison and unstable conditions

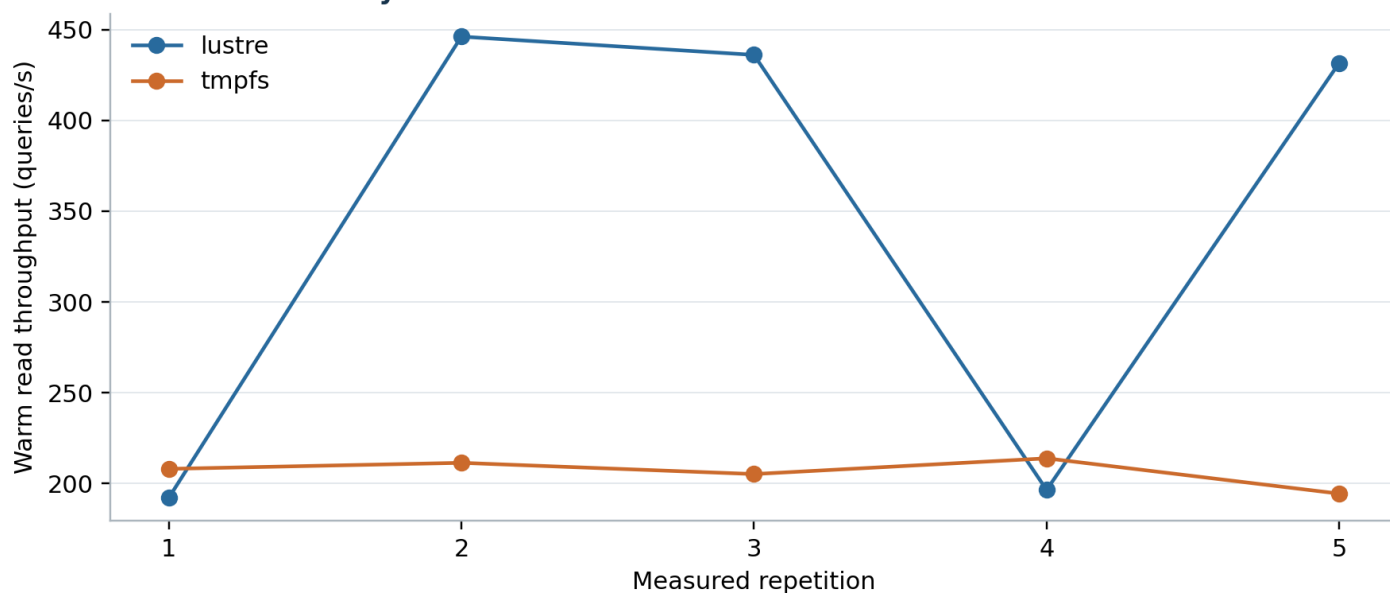
SQLite execution-environment comparison (descriptive)



Runtime ratios are descriptive. Native uses Python 3.11.7 / SQLite 3.44.2; Shifter uses Python 3.11.15 / SQLite 3.46.1. They run on different nodes and allocations. Their core worker, workload and backend hashes match, but the controller changed. Observed differences cannot be assigned solely to container overhead.

Shifter mean write throughput is lower in 11 of 12 SQLite conditions; the largest deficit is 15.6%. Telemetry on Lustre at 64 clients is 1.3% higher. Reads show differences in both directions. No causal runtime confidence interval is reported.

InfluxDB telemetry reads at 16 clients



InfluxDB telemetry, Lustre, 16 clients: arithmetic mean 340.48 queries/s; sample SD 133.54; median 431.62. The mean is sensitive to the slow repetitions. No observation was removed and no cause is inferred from this archive alone.

Write tail latency also rises with concurrency. Native SQLite metadata median trial p99 on tmpfs increases from 2.91 ms (1 client) to 4,339.14 ms (64 clients). These are batch latencies, not per-record latencies; full tail-latency figures and tables are in the accompanying package.

Methods, provenance and next experiments

Statistics and reproducibility

Warmups are excluded by their recorded phase, leaving five observations per condition. Arithmetic means, sample SD, medians, ranges and CV are exported. Mean intervals use mean $\pm t(0.975, 4) \times \text{SD}/\sqrt{5}$. No outliers are removed.

Storage and concurrency ratio intervals pair conditions by repetition within each allocation. For log ratios z , the reported geometric estimate is $\exp(\text{mean}(z))$; interval endpoints are $\exp(\text{mean}(z) \pm t(0.975, 4) \times \text{SD}(z)/\sqrt{5})$. Interaction contrasts subtract the 1-client log storage ratio from the 64-client ratio. Pairing uses the saved randomized repetition blocks; trials are not simultaneous.

Uncertainty scope: these model-based, pointwise intervals assume the five blocks adequately represent within-allocation variability. They are not adjusted for multiple comparisons, and do not cover between-node, between-day or between-allocation variation. Five trials in one job are not five independent allocations.

Source inventory

Deployment / workload	Slurm job	Node	Engine version
SQLite native / metadata	59115833	nid005486	3.44.2
SQLite native / telemetry	59115834	nid005772	3.44.2
SQLite Shifter / metadata	59132736	nid005384	3.46.1
SQLite Shifter / telemetry	59132737	nid005415	3.46.1
PostgreSQL / metadata	59136228	nid004148	16.12
PostgreSQL / telemetry	59136230	nid004158	16.12
InfluxDB / metadata	59188702	nid004584	2.9.1
InfluxDB / telemetry	59188703	nid004722	2.9.1

Input: ipdps-pilot-analysis.zip. Full SHA-256 is recorded in validation.json. CSV values reconcile with saved configurations, plans and selected metadata. Source-hash manifests are compared, but source bytes and individual request samples are not in this archive. Server-profiling runs are separate and are not pooled into these throughput estimates.

Next experiments that address specific gaps

1. Add independent allocation repeats with declared blocking and CPU affinity. **2.** For matched read comparisons, rerun SQL with the same query count and selection protocol as InfluxDB, or explicitly study query-duration sensitivity. **3.** Align write boundaries and document durability semantics before engine rankings. **4.** Use matching Python/SQLite builds for a controlled runtime comparison. **5.** Add mixed workloads and workload sizes, then evaluate a placement model on held-out allocations.